K-means clustering

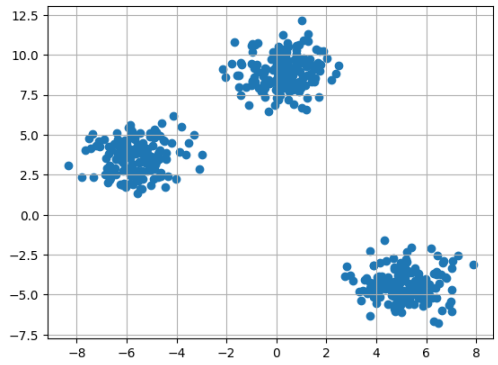
From geometric blobs to wine chemistry

A six-page guide with two executable notebooks

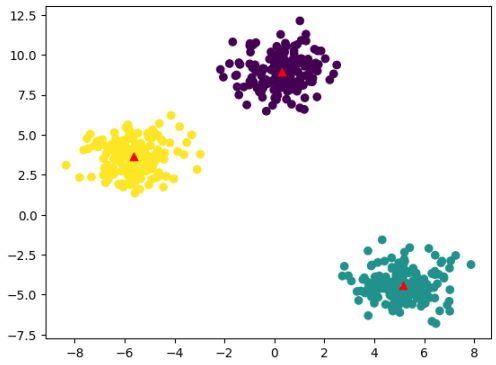
1. Start with a picture

Suppose a row records two coordinates of a point. A useful first question is whether nearby points can share a representative location. K-means answers this by learning K centers and assigning each row to one center. The same idea extends to rows with many measurements.

Reading connection. GeeksforGeeks introduces K-means through a generated point cloud: choose centers, assign points by distance, replace centers by group averages, and repeat. Its worked illustration supplies the two source images below [1].



Before grouping: an unlabeled point cloud.



After grouping: colors indicate assigned groups.

Figures reproduced from GeeksforGeeks, “K means Clustering – Introduction” [1]. Original captions: “Clustering dataset” and “K-means Clustering.” Images are resized; their rights remain with the respective rights holders. Direct image links are in ATTRIBUTION.md.

Clustering and classification. In supervised classification, training rows have target labels such as a known cultivar. Here the fitting algorithm sees only features. Group IDs 0, 1, 2 carry no ordered or biological meaning. Swapping those IDs leaves the grouping unchanged. Assigning a new point to a learned group is possible, but does not turn that group into a known class.

Learning route.

1. Work through the objective and a numerical update on page 2.
2. Run `01_geometric_blobs.ipynb`: inspect decision regions, optimization, and failure cases.
3. Run `02_wine_chemical_profiles.ipynb`: choose a distance, interpret groups, and audit them.

Both notebooks contain saved outputs, explanations before code, and exercises with runnable reference solutions. They use locally generated data or scikit-learn’s bundled Wine data. No runtime data download is needed.

Key idea. A cluster is a summary of similarity under a chosen distance. Its usefulness depends on what the measurements mean and how they are scaled.

2. What does K-means minimize?

Let $X \in \mathbb{R}^{n \times p}$ contain one observation x_i per row. For K centers $\mu_1, \ldots, \mu_K$, choose assignments $z_i \in \{1, \ldots, K\}$ to minimize

$$\boxed{W = \sum_{i=1}^{n} \|x_i - \mu_{z_i}\|_2^2 = \sum_{k=1}^{K} \sum_{i:z_i=k} \sum_{j=1}^{p} (x_{ij} - \mu_{kj})^2.}$$

W is *inertia*, or within-cluster sum of squares [2]. Squaring makes large errors costly: one distance of 10 costs as much as one hundred distances of 1.

Assignment. With centers fixed, each row independently chooses its least expensive center:

$$z_i \leftarrow \arg\min_k \|x_i - \mu_k\|^2.$$

The square root in Euclidean distance is unnecessary when comparing distances, because it preserves order.

Update. With assignments fixed, differentiate one group's contribution with respect to its center. For a nonempty group C_k,

$$\frac{\partial W}{\partial \mu_k} = 2 \sum_{i \in C_k} (\mu_k - x_i) = 0 \quad \Longrightarrow \quad \boxed{\mu_k \leftarrow \frac{1}{|C_k|} \sum_{i \in C_k} x_i.}$$

Thus the representative is an arithmetic mean. It can lie between observations. An empty group has no mean; an implementation needs an explicit repair or retention policy.

A complete update by hand. Use four 2D points and two starting centers:

$$x_1 = (0,0), \quad x_2 = (0,2), \quad x_3 = (8,0), \quad x_4 = (8,2), \qquad \mu_1 = (0,0), \quad \mu_2 = (8,0).$$

Point	Distance squared to μ_1	Distance squared to μ_2	Group
$(0,0)$	0	64	1
$(0,2)$	4	68	1
$(8,0)$	64	0	2
$(8,2)$	68	4	2

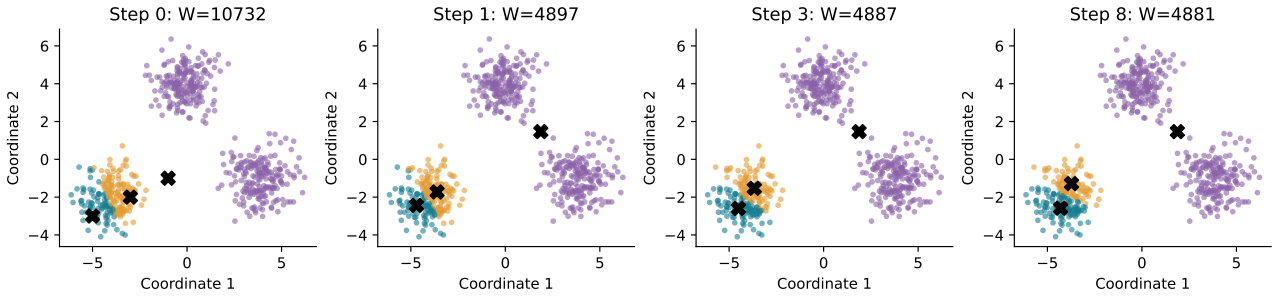
Initially $W = 8$. The updated means are $(0,1)$ and $(8,1)$, making every squared residual 1. Therefore $W = 4$. Assignments remain unchanged on the next pass, so these centers are stable. The boundary is the perpendicular bisector $x = 4$; points on it require a tie rule.

Why repeat? A center update can move the boundaries, changing the next assignments. Each exact assignment or update step cannot increase W. This does not establish a global optimum: alternating minimization can stop at a solution determined by its initialization [2].

Key idea. K-means alternates two easy subproblems. A low objective is a geometric fit criterion; it does not, by itself, validate the meaning of a group.

3. Watch the algorithm move

The first notebook uses 600 generated observations in two coordinates, with centers $(-4, -2)$, $(0, 4)$, $(4, -1)$ and isotropic noise SD 0.85. The coordinates share units, so this example preserves their original distance geometry.



Original experiment and plot: notebook 1. Crosses are current centers; each panel's colors are nearest-center assignments at that step. W is measured with those centers.

Trace the calculation. Inertia never increases, and the final result agrees with scikit-learn from the same starting centers. Both end at $W = 4880.9$: two centers divide one blob while the third joins the other two. The later multi-start fit reaches $W = 857.5$. A correctly implemented loop can therefore converge to a poor grouping. The teaching loop's empty-group policy retains the old center.

A practical fit. Separately from the teaching trace, use K-means++ seeding and repeated starts. K-means++ spreads candidate seeds using distance; it is not a guarantee of finding the best partition. Twenty starts improve the opportunity to find a low-inertia result [2,3].

```
model = KMeans(n_clusters=3, init="k-means++",
               n_init=20, random_state=23).fit(X)
groups = model.labels_
centers = model.cluster_centers_
new_groups = model.predict(new_points)
```

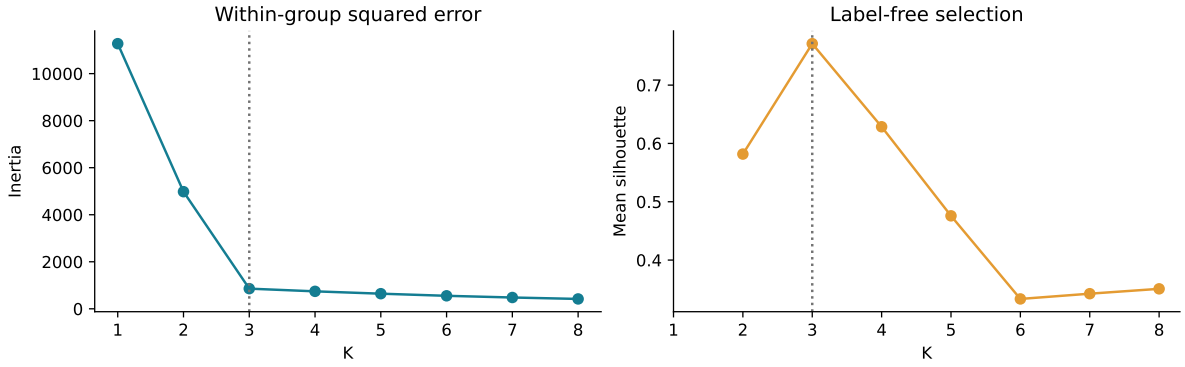
API member	Interpretation [3]
`labels_`	Group IDs for the fitting observations.
`cluster_centers_`	An array with K rows and p feature coordinates.
`inertia_`	Sum of squared residuals to assigned centers.
`predict(X_new)`	Nearest-center IDs, holding the fitted centers fixed.
`transform(X_new)`	Distance to every center; these are not probabilities.

Reproducibility and stopping. Fixing `random_state` makes this experiment repeatable within a software environment. Explicit `n_init` avoids dependence on changing defaults. Practical solvers also stop by a tolerance or iteration limit; near convergence, reported centers and means may differ slightly. The worked examples check their agreement numerically.

Key idea. A typical assignment pass costs about $O(nKp)$ operations. Multiple starts and iterations multiply that work; the small datasets here are suitable for a CPU.

4. Geometric blobs: select K and inspect the regions

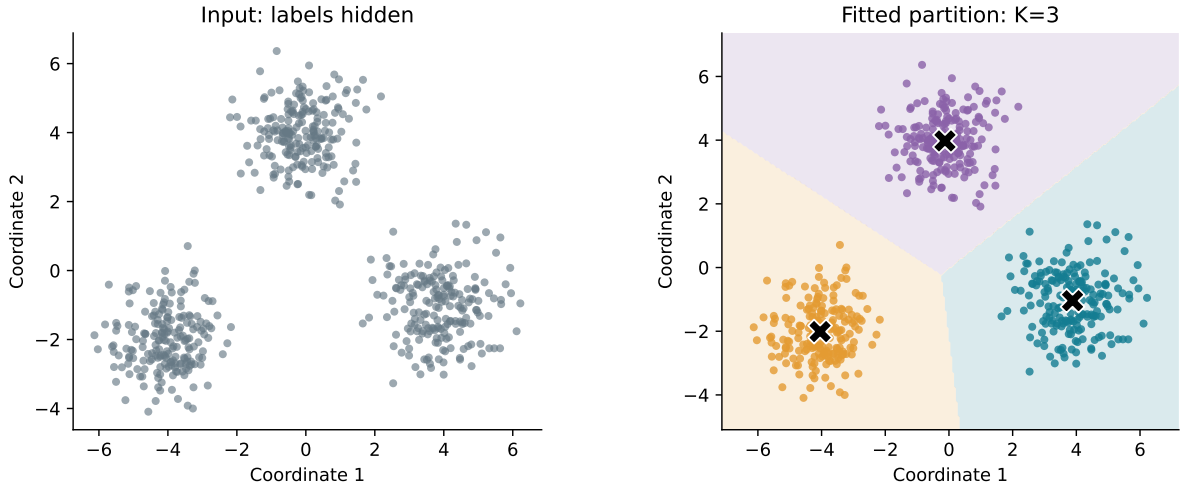
Choosing K is a separate decision from finding centers. The notebook scans K=1 to 8 and evaluates inertia plus the mean silhouette. It selects the largest silhouette among K=2 to 8, breaking ties toward the smaller K. Generating-component labels do not enter this choice.



Silhouette [5]. For observation i, let a_i be its average distance to the other rows in its group, and b_i the smallest average distance to another group. Then

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}.$$

Values near 1 indicate compact, separated membership; values near zero indicate a boundary; negative values suggest another group is closer on average. The score requires at least two groups and fewer groups than observations. Singleton observations receive zero by convention.



Original experiments and plots: notebook 1. Tinted regions show assignment of locations on a grid; crosses show learned centers. The grid does not represent additional fitting observations.

Executed result. Selected K=3, mean silhouette 0.771, inertia 857.5. After selection, adjusted Rand index (ARI) against component membership is 1.000. ARI compares same-group relationships and is unchanged by renaming IDs; 1 means identical partitions, while values near 0 indicate chance-level agreement under its reference model [6].

Read the curves carefully. The best possible inertia cannot increase as K grows; additional centers can reduce error even after useful groups have been represented. An elbow can be ambiguous. A favorable silhouette also evaluates the selected metric's geometry, so use domain interpretation and sensitivity checks alongside it.

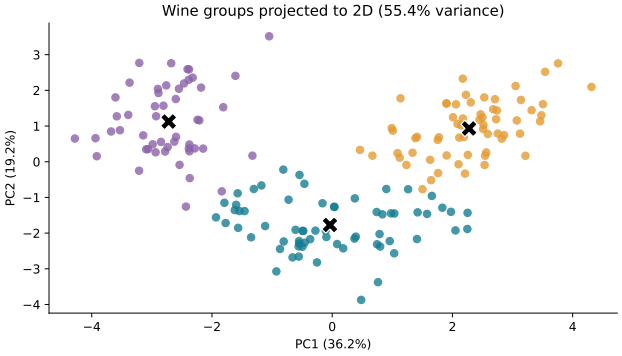
5. Real measurements: Wine chemical profiles

The UCI Wine dataset contains 178 samples and 13 chemical features from three cultivars [8]. The second notebook loads the scikit-learn packaged copy [9]. It uses every observation for descriptive grouping and reserves cultivar labels for a final audit. There is no test-accuracy claim.

Choose comparable feature contributions. A proline difference of 100 adds 10,000 to raw squared distance; an alcohol difference of 1 adds 1. We choose standardized coordinates $z_{ij} = (x_{ij} - \bar{x}_j)/s_j$, using each column's population SD [4]. This gives

$$\|z_i - z_\ell\|^2 = \sum_j \frac{(x_{ij} - x_{\ell j})^2}{s_j^2}.$$

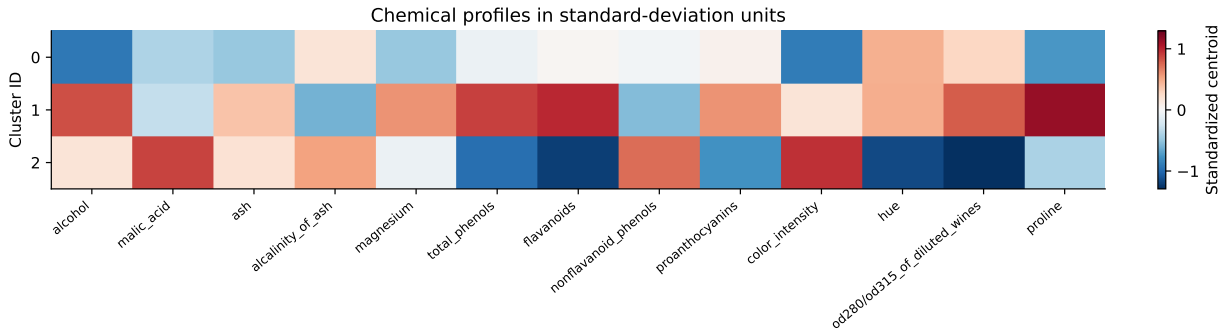
Standardization changes the metric. It gives comparable variance weighting, not equal chemical importance. Correlated features can still repeat information. Raw and standardized inertias have different units and cannot be directly ranked.



Executed results
Selected K: 3
Silhouette: 0.285
Cultivar ARI, scaled: 0.897
Cultivar ARI, raw at same K: 0.371
Two-PC variance retained: 55.4%
Mean subsample ARI: 0.990
Minimum subsample ARI: 0.947

Original experiment and plot: notebook 2. K-means fits all 13 standardized features. PCA supplies only a 2D view; projected distances omit variation in the other components [7].

Describe the centers. Standardized centroids show high and low measurements relative to the overall mean. Inverse-transforming them returns the original measurement scales. The notebook verifies that those inverse-transformed centers equal the original-unit group means.



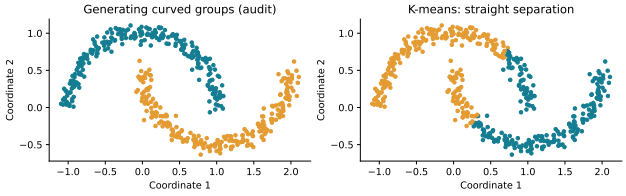
Original plot: notebook 2. Red is above the overall mean; blue is below it. IDs are arbitrary.

Audit without changing the selection. A contingency table compares frozen cluster membership with cultivars. Ordinary accuracy on raw IDs is inappropriate because IDs can be permuted. For stability, refit both scaler and K-means on twenty 80% subsamples, fix K, predict all original rows, and compare each partition with the full fit. Stable profiles remain descriptive; they are not proof of biological categories or evidence of performance on future wines.

6. Limits, practice, and sources

Geometry can defeat the model. Each center owns a convex nearest-center region. Two curved moons need not match such a partition. In the notebook's K=2 moon experiment, ARI is 0.241, even though the solver still returns centers and group IDs.

Other concerns. Outliers pull arithmetic means; mixed scales change distance; unequal densities or elongated groups can lead to misleading partitions. Every observation, even one far outside the fitted cloud, receives an assignment.



Original comparison from notebook 1.

Practice prompts. The notebooks include executable reference solutions.

1. Increase blob noise SD from 0.85 to 1.8. Repeat selection, then inspect ARI. Explain the difference.
2. Multiply one coordinate by 20. Compare raw and standardized clustering at fixed K=3.
3. Describe each Wine centroid using its three strongest standardized deviations.
4. Remove proline, refit preprocessing and centers at the selected K, and compare partitions.

A workflow to retain. Define the row and feature meanings; choose a defensible distance; fit several starts over candidate K values; inspect compactness, profiles, and stability; then use any external labels for a clearly identified audit. Apply the fitted preprocessing to future rows.

References and attribution

All online references accessed 27 October 2026. The short reading connection on page 1 is paraphrased from [1]. The two page-1 images are reproduced from that article. Remaining prose, derivations, code, experiments, and figures were prepared for this supplement, with technical references below. `ATTRIBUTION.md` records direct image URLs and checksums.

[1] GeeksforGeeks. K means Clustering – Introduction. Copyright remains with the respective rights holders; attribution does not change the images' license.

[2] scikit-learn. Clustering user guide: K-means.

[3] scikit-learn. KMeans API reference.

[4] scikit-learn. StandardScaler API reference.

[5] scikit-learn. Silhouette score API reference.

[6] scikit-learn. Adjusted Rand score API reference.

[7] scikit-learn. PCA API reference.

[8] Aeberhard, S. & Forina, M. (1992). *Wine* [Dataset]. UCI Machine Learning Repository. doi:10.24432/C5PC7J. CC BY 4.0.

[9] scikit-learn. load_wine API reference. Packaged copy: standardized feature names and zero-based target codes. Analysis transformations: standardization and PCA.